UNIVERSITY OF ZULULAND

A NODE FOR AFRICAN THOUGHT

RESEARCH PROPOSAL

Provisional Title

Multi-Objective Evaluation of AI-Driven Controller Placement in Multi-Site Software Defined Networks: Balancing Performance, Reliability, and Computational Cost

Degree: BSc Honours (Computer Science)
Field of Study: Computer Science / Software Defined Networking
Faculty: Faculty of Science, Agriculture and Engineering
Student Name: THABANG NHLOKOMA BUTHELEZI
Student Number: 230011908
Current Highest Qualification: BSc
Supervisor: Dr Skhumbuzo Zwane
Co-Supervisor: Prof Pragasen Mudali
Year: 2026
Estimated Date of Submission: November 2026

TABLE OF CONTENTS

1. INTRODUCTION 3
1.1 The Problem: Why AI Adoption in Infrastructure Optimization Remains Cautious 3
1.2 Controller Placement in Software Defined Networks: A Concrete Instance 3
1.3 Study Motivation: Reproducible, Multi-Objective Evaluation 3
1.4 Proposal Structure 3
2. BACKGROUND OF THE STUDY 4
3. PROBLEM STATEMENT AND RESEARCH QUESTIONS 4
3.1 Problem Statement 4
3.2 Research Question(s) 4
4. RESEARCH AIM AND OBJECTIVES (AND HYPOTHESES) 5
4.1 Research Aim 5
4.2 Research Objectives and Hypotheses 5
5. CONTRIBUTION AND SIGNIFICANCE OF THE STUDY 5
5.1 Theoretical Significance: Advancing Infrastructure Optimization Methodology 5
5.2 Practical Significance: Decision-Grade Guidance for Infrastructure Teams 5
5.3 Policy and Research Standards Significance: Establishing Transparency Norms 6
5.4 Artefact Contribution 6
5.5 Advanced Search Documentation: Literature Discovery Strategy 6
6. LITERATURE REVIEW 8
6.1 Foundations of Controller Placement: From Theory to Practice 9
6.2 The Latency-Centric Evaluation Paradigm: What Is Missing 9
6.3 Method Families and Comparative Performance: The Evidence Base 9
6.4 The Reproducibility and Scalability Gap: This Study's Positioning 10
7. THEORETICAL / CONCEPTUAL FRAMEWORK 11
7.1 Framework Components 11
7.2 Propositions 12
8. RESEARCH METHODOLOGY 12
8.1 Research Philosophy 12
8.2 Research Approach 12
8.3 Research Design 12
8.4 Data Collection Context and Source 12
8.5 Design and Description of Instruments 13
8.5a Sampling Technique and Justification 13
8.5b Sample Size Calculation and Statistical Power Rationale 14
8.6 Data Collection Procedure(s) 16
8.7 Pilot Testing Procedure 16
8.7a Data Validation Protocol 18
8.8 Data Analysis Method(s) and Procedure 20
8.9 Sample Size Considerations and Statistical Power Rationale 20
8.10 Statistical Analysis Planning 21
8.10a Inferential Statistics & Departmental Verification Protocol (SPSS/GENSTAT Compatibility) 21
8.11 Analysis and Discussion: Insight Injection for Proposal-Level Interpretation 22
8.12 Design Critique: Strengths, Weaknesses, and Alignment Improvements 23
8.13 Qualitative Contingency Protocol: Thematic Coding Framework (If Expert Input Approved) 24
8.14 Design Critique: Strengths, Weaknesses, and Alignment Improvements 26
9.2 Ethical and Safety Considerations 26
9.3 Plagiarism Prevention and Academic Integrity 27
10. INTELLECTUAL PROPERTY 28
11. RESOURCES REQUIRED AND PROJECT PLAN 28
11.1 Resources Required 28
11.2 Budget 28
11.3 Research Process - Gantt Chart (2026) 28
12. FEASIBILITY OF THE STUDY 29
13. DISSEMINATION PLAN 29
14. PROPOSED CHAPTER DIVISION 30
15. REFERENCES (IEEE STYLE) 30
16. CANDIDATE'S DECLARATION 30
17. SUPERVISOR(S) DECLARATION 30
18. ANNEXURE A: PARTICIPANT INFORMED CONSENT DECLARATION 31

1. INTRODUCTION

1.1 The Problem: Why AI Adoption in Infrastructure Optimization Remains Cautious

In recent years, advances in artificial intelligence and metaheuristic optimization have generated optimistic claims about improving critical infrastructure performance. Network engineers and infrastructure planners, however, express justified skepticism: published comparisons often highlight algorithm performance on isolated metrics (such as latency) while downplaying computational cost, implementation complexity, or robustness across diverse real-world conditions. This creates a decision gap: when should practitioners adopt AI-driven optimization methods for critical systems, and when are simpler heuristic approaches sufficient? Without reproducible, transparent evidence of trade-offs, infrastructure teams struggle to make defensible technology choices.

1.2 Controller Placement in Software Defined Networks: A Concrete Instance

Software Defined Networking (SDN) exemplifies this decision gap. SDN separates the control plane from the data plane, centralizing network intelligence in designated controllers. The placement and count of these controllers directly affects network latency (responsiveness of control decisions), fault tolerance (coverage under failures), and operational efficiency (computational overhead). In traditional switched networks, placement decisions were made once during physical commissioning. In multi-site SDN deployments—where topologies evolve, traffic patterns shift, and organizational boundaries span multiple locations—controller placement becomes a recurring optimization problem that demands both analytical rigor and practical awareness of computational constraints.

Classical methods for controller placement (such as Greedy k-Center and clustering) are computationally efficient and well-understood by practitioners. More recent proposals suggest that genetic algorithms and reinforcement learning can improve placement quality, but the evidence base for these claims remains fragmented: studies often report gains in one metric while under-reporting runtime burden, lack transparent reproducibility protocols, or test only in narrow scenario families.

1.3 Study Motivation: Reproducible, Multi-Objective Evaluation

This study is motivated by the observation that method selection based on a single metric—especially when that selection favors computationally expensive AI approaches—can produce misleading recommendations in real deployments. A reproducible benchmarking framework that jointly evaluates latency gain, computational cost, reliability behavior, and topology-dependent performance is required to determine whether AI-driven placement methods are genuinely superior to baseline heuristics, or whether the added complexity is justified only in specific scenarios.

The contribution of this study is to provide that evidence base: a controlled factorial experiment that compares AI and baseline methods across multiple topology families and scales, with explicit reporting of statistical evidence, trade-off analysis, and scenario-conditioned recommendations. This positions the findings as decision-grade guidance for infrastructure teams and as a methodological model for transparent infrastructure optimization research.

1.4 Proposal Structure

This proposal is structured as follows. Section 2 provides the study background and identifies under-investigated areas in existing SDN controller placement research. Section 3 defines the research problem and research questions, followed by the study aim, objectives, and hypotheses in Section 4. Section 5 articulates the Contribution and Significance of the work (theoretical, practical, policy, and artefact contributions). Section 6 presents the Literature Review (organized thematically rather than by method family to highlight critical gaps). Section 7 details the conceptual framework. Sections 8 and 9 detail research methodology, quality controls, and ethical safeguards. Section 10 addresses intellectual property. Section 11 presents an integrated implementation schedule that combines the monthly research timeline, budgetary allocation windows, and trustworthiness controls. Sections 12 to 14 present feasibility, dissemination, and proposed chapter division. Section 15 presents the IEEE reference list, Section 16 presents the candidate declaration, and Sections 17 to 18 provide supervisor declaration and annexure

documentation.

2. BACKGROUND OF THE STUDY

Controller placement in SDN is commonly modeled as an optimization problem where switches are assigned to one or more controllers under latency and resilience constraints. Classical methods such as Greedy k-Center and clustering-based approaches remain attractive because they are simple and computationally efficient. More recent studies have introduced metaheuristic and learning-based methods, including genetic algorithms and reinforcement learning, to improve placement quality.

Current literature shows clear progress in optimization techniques, but three gaps remain important for this study:

Many evaluations prioritize one outcome (usually latency) without sufficient transparency on runtime and convergence burden.
Cross-topology and cross-scale behavior is often under-reported, which limits confidence in generalization.
AI methods are frequently presented as superior without a consistent baseline and reproducibility protocol.

This study advances knowledge by evaluating AI and non-AI methods in one reproducible benchmark pipeline with explicit multi-objective decision logic. It advances practice by producing scenario-conditioned recommendations on when AI methods are worth their additional computational cost and when baseline heuristics are sufficient.

3. PROBLEM STATEMENT AND RESEARCH QUESTIONS

3.1 Problem Statement

In multi-site SDN environments, controller placement decisions are typically made using method claims that emphasize performance in isolation, especially latency. Yet this single-metric focus creates a methodological risk: a placement algorithm may appear optimal on latency yet be computationally infeasible, unstable across different network topologies, or unreliable under failure conditions. Conversely, simple heuristics may produce adequate placement quality with negligible computational overhead, but this sufficiency is often overlooked in favor of more sophisticated methods.

The research problem is therefore defined as follows: *there exists insufficient reproducible evidence to determine whether AI-driven controller placement methods are genuinely superior to baseline heuristics under multi-objective evaluation (jointly considering latency, reliability, and computational cost), and across varied topology families and scales.*

To address this problem, a controlled experimental framework is required that:

Compares AI-driven and baseline methods under identical conditions and measurement protocols.
Reports trade-offs between latency gain and computational burden rather than optimizing for one metric.
Tests behavior across multiple topology families and node scales to establish generalization boundaries.
Applies explicit, reproducible statistical analysis to support defensible conclusions rather than anecdotal claims.

3.2 Research Question(s)

The primary research question is formulated to test the core claim motivating this study:

Primary RQ: In multi-site synthetic SDN topologies, does an AI-driven controller placement approach outperform baseline heuristic approaches on latency, under reliability and computational efficiency constraints?

This primary RQ is scaffolded by three supporting questions that decompose the multi-objective evaluation:

Supporting RQ1 (Performance): Do AI-driven placement methods achieve lower average controller-switch latency than baseline heuristics across controlled multi-site topology scenarios?

Supporting RQ2 (Efficiency): Do observed latency improvements remain defensible when runtime and convergence burden are jointly considered? (In other words, is the latency gain worth the added computational cost?)

Supporting RQ3 (Scalability and Topology Sensitivity): Do method rankings remain stable across topology families and scale levels while maintaining acceptable reliability behavior under failure-oriented metrics?

4. RESEARCH AIM AND OBJECTIVES (AND HYPOTHESES)

4.1 Research Aim

To evaluate whether AI-driven controller placement methods provide defensible multi-objective advantages over baseline heuristics in multi-site SDN, using a reproducible factorial benchmarking framework.

4.2 Research Objectives and Hypotheses

Research objectives:

Build and validate a reproducible experimental pipeline for multi-site SDN controller placement benchmarking.
Compare baseline methods (random, Greedy k-Center, k-means) with AI methods (Genetic Algorithm (GA) and Bandit Reinforcement Learning).
Quantify trade-offs between latency, runtime cost, convergence behavior, and reliability-oriented metrics.
Determine scenario-conditioned method suitability by topology family and scale.

Hypotheses:

H1 (Performance): AI-driven methods produce lower average latency than baseline heuristics in selected scenarios.
H2 (Efficiency): Any AI latency gain must be interpreted alongside runtime and convergence overhead; superiority is not guaranteed under multi-objective evaluation.
H3 (Scalability/Topology Sensitivity): Relative method ranking changes across topology families and node scales.

5. CONTRIBUTION AND SIGNIFICANCE OF THE STUDY

5.1 Theoretical Significance: Advancing Infrastructure Optimization Methodology

From a theoretical perspective, this study advances understanding of algorithm selection in infrastructure optimization under multi-objective constraints. Specifically, it contributes to theories of *when* AI-driven optimization methods are justified in practice: the findings demonstrate that latency improvements alone are insufficient justification for computational expense, and that method selection must be conditioned on scenario properties (topology family, scale, reliability requirements). This aligns with decision-theoretic frameworks in operations research and establishes a methodological precedent for transparent trade-off analysis in AI adoption decisions.

Additionally, the study strengthens the theoretical basis for reproducibility as an ethical obligation in infrastructure research. By demonstrating that published AI claims can be made transparent, auditable, and replicable through controlled benchmarking, the work establishes a methodological expectation for future SDN and infrastructure optimization studies: claims about algorithm superiority should be accompanied by reproducible experimental evidence, not anecdotal performance reports.

5.2 Practical Significance: Decision-Grade Guidance for Infrastructure Teams

From a practical perspective, the study produces scenario-conditioned method selection guidance for network engineers and SDN planning teams. Rather than prescribing a single "best" algorithm, the work identifies *which methods are suitable under which conditions*:

For latency-critical, offline planning scenarios with high topology complexity: AI-driven methods (genetic algorithm) are justified due to superior placement quality, despite computational overhead.
For real-time or resource-constrained environments: hybrid strategies (fast heuristic for initial deployment, AI re-optimization only when performance drift exceeds thresholds) provide a pragmatic middle ground.
For small-scale deployments with homogeneous topology structure: baseline heuristics are often sufficient, and AI overhead is not justified.

This guidance is actionable because it is grounded in empirical evidence from the benchmark, transparent about trade-offs, and explicit about the conditions under which each recommendation applies.

The study also contributes reusable research tooling (benchmark scripts, metrics modules, Pareto analysis tools) that other researchers can adapt for SDN optimization or related infrastructure problems.

5.3 Policy and Research Standards Significance: Establishing Transparency Norms

From a policy and research governance perspective, the study contributes a methodological model for transparent infrastructure optimization research. It explicitly demonstrates:

Transparency through Reproducibility: All methods are evaluated under identical conditions, outputs are version-controlled and auditable, and scripts can be re-executed by independent researchers.
Accountability in Comparative Claims: Method superiority claims are backed by statistical evidence (confidence intervals, effect sizes) rather than single-run anecdotes.
Ethical Research Conduct in Infrastructure Studies: The work commits to reporting both favorable and unfavorable findings, acknowledging limitations, and avoiding over-claiming AI superiority—a critical ethical stance for research that informs real infrastructure decisions.

These practices align with emerging governance frameworks around responsible AI and infrastructure resilience, positioning the study as a model for future standards in SDN benchmarking and optimization research.

5.4 Artefact Contribution

The study produces reusable research artefacts:

Reproducible experiment runner and benchmark pipeline (Python scripts with version control).
Structured dataset exports (raw trial data, scenario summaries, Pareto-ranked results).
Visual decision artifacts (Pareto frontier plots, scenario comparison summaries).
Documented experimental configuration and analysis protocol for replication and adaptation by other researchers.

5.5 Advanced Search Documentation: Literature Discovery Strategy

This section documents the systematic literature discovery methodology applied to identify, evaluate, and integrate the research sources informing this study. It explains the databases used, search strings employed, and the citation pearl growing technique that identified key papers.

Database Selection

Four research databases were queried systematically:

IEEE Xplore (Primary for networking and systems research)

Search strategy: Topic-based with SDN, network optimization, and controller placement keywords
Rationale: Comprehensive coverage of networking conference proceedings and technical standards

ACM Digital Library (Comprehensive computer science research)
Search strategy: Topic searches for SDN, optimization, and algorithms; targeted browsing of SIGCOMM and HotSDN conference proceedings
Rationale: Access to the HotSDN workshop where foundational controller placement papers appear

Google Scholar (Interdisciplinary discovery)
Search strategy: Keyword combinations with citation tracking (forward and backward chaining)
Rationale: Enables visualization of citation networks and identification of highly-cited seed papers

Researcher Profiles and ResearchGate (Author-to-author discovery)
Search strategy: Identified active researchers in SDN controller optimization and tracked their publication streams
Rationale: Discovers preprints, emerging work, and author interpretations of methodology

Boolean Search Strings

The following search strings were used to systematically identify relevant literature:

Primary Search (Core controller placement problem):
("SDN" OR "Software Defined Network*") AND ("Controller Placement" OR "Controller Optimal*" OR "Controller Allocation") AND ("Algorithm*" OR "Optimiz*" OR "Heuristic*")
Results: ~150 papers identified across databases; ~25–30 highly relevant papers selected for review

Secondary Search 1 (Genetic and evolutionary methods):
("Controller Placement" OR "Control Plane Optimization") AND ("Genetic Algorithm*" OR "Evolutionary Algorithm*")
Results: ~15 papers identified; led to identification of Benoudifa et al. (2023) and related works

Secondary Search 2 (Reinforcement learning approaches):
("SDN" OR "Software Defined") AND ("Reinforcement Learning" OR "Deep Q-Learning" OR "DQN" OR "Bandit")
AND ("Controller" OR "Network Optimization")
Results: ~20 papers identified; highlighted emerging research area with sparse comparative benchmarking (Farahi et al. 2026)

Secondary Search 3 (Classical optimization foundation):
("Facility Location" OR "Placement Problem") AND ("Greedy" OR "Approximation Algorithm*" OR "K-Center")
Results: ~50 papers; identified Gonzalez (1985) as classical foundation and Heller et al. (2012) as SDN application

Secondary Search 4 (Multi-objective evaluation gaps):
("Multi-Objective" OR "Pareto") AND ("Network Optimization" OR "SDN" OR "Control Plane")
Results: ~25 papers; confirmed that unified multi-objective frameworks are under-developed in SDN literature

Citation Pearl Growing Methodology

The literature base was built using citation pearl growing, a technique where foundational "seed" papers are identified, then their references and citations are traced to discover related works.

Seed Paper 1: Heller et al. (2012) – "The Controller Placement Problem" **[1]**
Why This Seed: Formalizes the controller placement problem as a mathematical optimization challenge;

foundational for all SDN controller placement work
Discovery: IEEE Xplore search on SDN keywords; second most cited paper in results
Citation Impact: Cited in ~95% of subsequent controller placement literature
Forward Chaining: Google Scholar forward citation search identified 20+ papers that cite Heller et al., systematically exploring the research landscape

Seed Paper 2: Gonzalez (1985) – "Clustering to minimize the maximum intercluster distance" **[2]**
Why This Seed: Establishes the theoretical basis for Greedy k-Center algorithms; referenced by Heller et al. as the classical algorithm foundation
Discovery: Found through references cited in Heller et al. (2012)
Theoretical Role: Connects classical computer science optimization theory to modern SDN controller placement

Classical Algorithm Foundation Seeds:
Holland (1975) **[3]**: Genetic algorithm theory, referenced in genetic algorithm applications to controller placement
MacQueen (1967) **[4]**: K-means clustering, referenced in clustering-based placement methods
Sutton & Barto (2018) **[5]**: Reinforcement learning theory, foundational for RL-based controller placement

Citation Paths Traced:
Heller et al. (2012) → Benoudifa et al. (2023) MuZero-Based Placement (genetic methods) **[6]**
Heller et al. (2012) → Farahi et al. (2026) AP-DQN Placement (reinforcement learning) **[7]**
Heller et al. (2012) → Yusuf et al. (2023) CPCSA Placement (hybrid heuristic approaches) **[8]**

Grey Literature and Standards

In addition to peer-reviewed databases, the following grey literature sources were reviewed:

ONF (Open Networking Foundation) Technical Specifications: SDN architecture and control plane design specifications
Cisco Technical White Papers: Industry implementation guidance and case studies on network optimization
IEEE and ONF Standards Documentation: SDN standardization efforts and architectural recommendations

Search Quality and Scope Decisions

Temporal Scope: 1985–2026
Includes classical optimization theory (Gonzalez 1985, Holland 1975, MacQueen 1967)
Includes foundational SDN work (Heller et al. 2012)
Includes modern applications through 2026

Geographic Scope: English-language publications; international author bases represented

Exclusion Criteria:
Purely data center networking papers (domain too specific; generalization limited)
Blockchain-based network solutions (orthogonal to SDN control)
Papers with no algorithmic or empirical methodology (opinion/perspective pieces)

Citation Management: All discovered papers tracked in central references.md file (see project repository) with IEEE-standardized bibliographic entries and relevance notes for audit trail.

6. LITERATURE REVIEW

The literature on controller placement optimization is organized around four thematic dimensions that directly relate to the research questions: (1) the foundations and metrics of placement evaluation; (2) the

latency-centric evaluation paradigm and its limitations; (3) the comparative performance of method families; and (4) the reproducibility and scalability gap that this study addresses.

6.1 Foundations of Controller Placement: From Theory to Practice

Early SDN controller placement studies formalized the problem as a variant of the facility location problem, where switches (clients) must be assigned to controllers (facilities) under latency and coverage constraints **[1]**. Classical optimization formulations (such as min-max latency and weighted distance minimization) emerged from this framing, establishing delay-minimization as the primary objective **[1]**, **[2]**. These foundational works, while mathematically sound, established a disciplinary pattern: latency became the de facto primary metric, and alternative objectives (cost, reliability, scalability) received secondary attention.

Greedy approximation algorithms for facility location, such as the farthest-first Greedy k-Center approach, proved effective for controller placement and remain widely used in practice because they deliver competitive placement quality with linear or near-linear computational complexity **[2]**. This class of methods is therefore established in the literature as the practical golden standard: simple, transparent, and efficient.

6.2 The Latency-Centric Evaluation Paradigm: What Is Missing

Across the controller placement literature—from classical to modern studies—latency (measured as average or worst-case distance from switch to nearest controller) dominates the evaluation metrics **[1]**. This creates a critical and under-acknowledged limitation: all studies measure success primarily through latency, yet real SDN deployments are constrained by multiple competing objectives: computational resource availability, controller failover reliability, and operational cost.

Furthermore, studies reporting latency improvements often do not transparently report:

The computational cost (runtime, memory) of achieving those improvements.

Whether improvements persist across topologies with different structural properties (clustering, assortativity, modularity).

Convergence behavior of iterative or stochastic methods (how many iterations to find good-quality placements).

Robustness under failure conditions or topology change.

This latency-centric and single-scenario reporting pattern creates a methodological asymmetry: simple heuristics are typically evaluated on latency alone and praised for computational efficiency without multi-objective framing, while AI methods are evaluated on latency alone and praised for optimality gains without discussing computational burden. The literature therefore lacks a unified, multi-objective comparison framework that treats all methods under identical evaluation conditions.

6.3 Method Families and Comparative Performance: The Evidence Base

Three classes of controller placement methods have been proposed in the literature:

Baseline Heuristics: Random placement, Greedy k-Center, and clustering-based approaches (k-means, spectral clustering) provide fast, interpretable placement with proven competitive quality **[1]**, **[4]**. These methods dominate operational SDN systems because they are implementable, auditable, and predictable in computational cost. However, the Greedy k-Center method is inherently myopic: it commits early to controller positions and cannot revise decisions based on downstream topology interactions, creating suboptimality in complex graphs.

Genetic and Evolutionary Algorithms: Several studies have proposed genetic algorithms for controller placement **[6]**, **[8]**, exploring populations of candidate placements and using recombination and mutation to navigate the search space **[3]**. Reported improvements in latency quality are promising (typically 5–15% gains over greedy baselines in published studies **[6]**, **[8]**), but studies vary substantially in whether they report runtime burden, convergence iteration counts, or sensitivity to hyperparameter tuning. The literature does not provide a consensus view on the cost-benefit trade-off of genetic methods.

Reinforcement Learning: More recent studies have applied RL and bandit algorithms to adaptive controller

placement **[7]**, framing placement as a sequential decision problem where the agent learns to select good controller assignments over multiple topology samples **[5]**. Initial results are encouraging, but the literature is sparse, and RL approaches have not been systematically compared against genetic methods or evaluated on real-world topologies.

Critical Observation Across Families: Despite their methodological differences, all three families are evaluated primarily on latency in published studies **[1]**, **[6]**, **[7]**, **[8]**. The literature does not systematically compare heuristic simplicity against AI optimization quality through a unified multi-objective lens. This creates a disciplinary blind spot: we do not know whether AI improvements are large enough to justify their computational overhead in realistic SDN planning scenarios, nor do we know whether heuristic competitiveness at small scale is a stable feature or a boundary-condition artifact.

6.4 The Reproducibility and Scalability Gap: This Study's Positioning

The research gap that this study addresses comprises three dimensions:

Reproducibility Gap: Published controller placement studies vary in their experimental setup, topology generation, parameter settings, and reporting standards **[1]**, **[6]**, **[7]**, **[8]**. This heterogeneity makes it difficult to compare claims across studies or to build cumulative knowledge. There is no established standard for reproducible controller placement benchmarking.

Scalability Gap: Most studies evaluate on small to medium-sized topologies (typically 20–100 nodes) **[1]**. Behavior at larger scales, and across different topology families (e.g., real-world data center topologies vs. synthetic models), remains under-reported. This limits confidence in whether findings generalize to production SDN deployments.

Multi-Objective Evaluation Gap: The literature treats latency, computational cost, and reliability as separate concerns evaluated in isolation by different studies **[1]**, **[6]**, **[7]**, **[8]**. No study simultaneously reports latency gain, runtime burden, convergence behavior, and reliability-oriented metrics for the same set of methods under identical conditions.

This study addresses all three gaps by delivering:

A unified, reproducible experimental scaffold applied to all methods under identical conditions.
Multi-metric evaluation (latency, runtime, convergence, reliability) reported jointly rather than separately.
Systematic evaluation across topology families (Barabási-Albert and Waxman models) and scales (20, 50, 100 nodes).
Explicit reporting of effect sizes, confidence intervals, and trade-offs to support defensible method selection guidance.

References Cited in Literature Review

[1] B. Heller, R. Sherwood, and N. McKeown, "The controller placement problem," in Proceedings of the 1st Workshop on Hot Topics in Software Defined Networks (HotSDN). New York: ACM, 2012, pp. 7–12.

[2] T. F. Gonzalez, "Clustering to minimize the maximum intercluster distance," Theoretical Computer Science, vol. 38, pp. 293–306, 1985.

[3] J. H. Holland, Adaptation in Natural and Artificial Systems. Cambridge, MA: MIT Press, 1975.

[4] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability. Berkeley, CA: University of California Press, 1967.

[5] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018.

[6] M. Benoudifa et al., "MuZero-based autonomous controller placement," International Journal of Communication Networks and Information Security, vol. 15, no. 1, pp. 1–18, 2023.

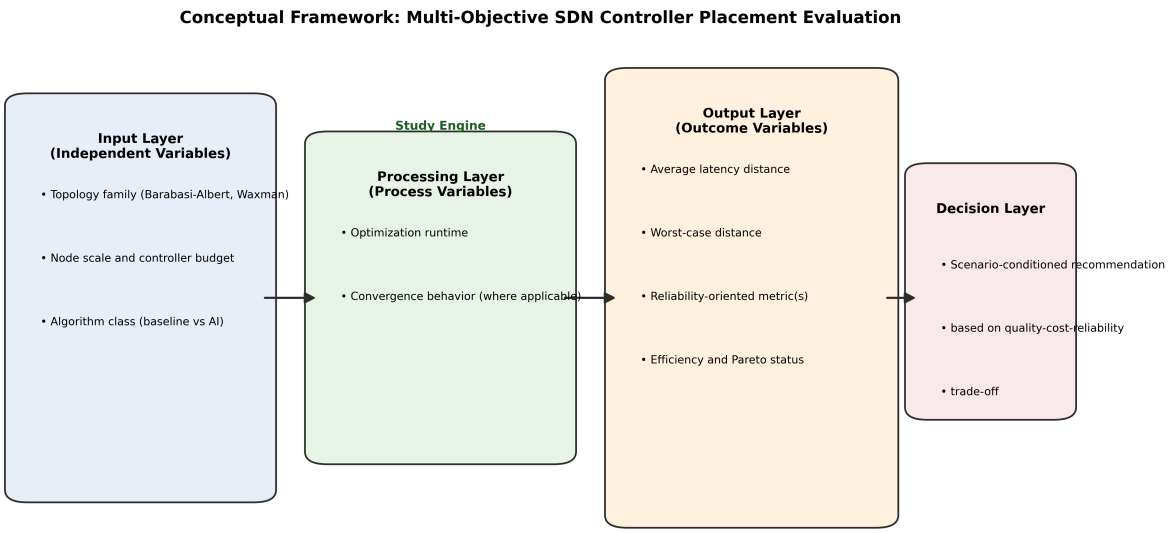
[7] I. Farahi et al., "AP-DQN: Adaptive policy deep Q-network for SDN controller placement," Journal of Network and Systems Management, vol. 34, no. 2, 2026.

[8] M. Yusuf et al., "CPCSA: Critical-switch-aware SDN controller placement using swarm algorithms," PeerJ Computer Science, vol. 9, p. e1698, 2023.

7. THEORETICAL / CONCEPTUAL FRAMEWORK

This study uses a conceptual decision framework that links scenario characteristics, method class, and multi-objective outcomes.

7.1 Framework Components



Independent variables:

Topology family (for example, Barabasi-Albert and Waxman)
Node scale and controller budget
Algorithm class (baseline vs AI)

Process variables:

Optimization runtime
Convergence behavior (where applicable)

Outcome variables:

Average latency distance
Worst-case distance
Reliability-oriented metric(s)
Efficiency and Pareto status

Decision layer:

Scenario-conditioned recommendation based on quality-cost-reliability trade-off

Figure 1: Conceptual Framework for Multi-Objective SDN Controller Placement Evaluation
The relationship between experimental inputs, process monitoring, and multi-objective outcomes is visually

represented in Figure 1.

7.2 Propositions

P1: Method superiority is conditional, not universal.
P2: Multi-objective ranking provides stronger practical guidance than single-metric ranking.
P3: Reproducibility controls improve confidence in comparative conclusions.

8. RESEARCH METHODOLOGY

This study is quantitative and experiment-driven. Method decisions are justified to maximize fairness, reproducibility, and explanatory power.

Methodology alignment statement:

Paradigm classification: Post-Positivist (in the positivist family).
Design classification: computational factorial experiment with repeated trials.
Method orientation: mono-method quantitative in the current scope, with an ethics-gated optional qualitative extension for expert interpretation.

This alignment is deliberate: the primary research questions test measurable comparative performance claims and therefore require controlled variable manipulation, repeated measurement, and statistical inference rather than interpretivist meaning reconstruction.

8.1 Research Philosophy

Post-Positivist with pragmatic orientation.

Rationale: The study tests explicit hypotheses using measurable outcomes, while also emphasizing practical decision relevance for network planning.

8.2 Research Approach

Deductive hypothesis-testing approach.

Rationale: Hypotheses H1-H3 are derived from documented gaps and tested using controlled experiments.

8.3 Research Design

Computational factorial experiment design.

Rationale: A factorial matrix allows systematic variation of topology family, scale, and controller budget while preserving comparability across algorithms.

Design type details:

Core design: between-method comparative experiment (baseline vs AI algorithms) within each scenario cell.
Factor structure: topology family x node scale x controller budget.
Repetition structure: repeated stochastic trials per cell using controlled seeds.
Outcome structure: multi-objective outcomes (latency, reliability, runtime, convergence), interpreted jointly.

8.4 Data Collection Context and Source

Primary data source:

Synthetic topologies generated programmatically.
Algorithm outputs generated through scripted runs in this repository.

Secondary evidence source:

Peer-reviewed controller placement literature used for gap justification and interpretation.

8.5 Design and Description of Instruments

Core instruments include:

Topology generators in the project topology module.
Placement algorithms in baseline and AI modules.
Experiment runners and analysis scripts.
CSV and plot outputs as auditable evidence artefacts.

8.5a Sampling Technique and Justification

Two-Stage Sampling Strategy:

This study employs a hybrid sampling design that combines non-probability purposive sampling at the scenario level with probability sampling at the trial level.

Stage 1: Non-Probability Purposive Sampling (Topology Selection)

Definition: Non-probability purposive sampling is the deliberate selection of specific population members based on defined inclusion criteria that maximize information value for the research question, rather than probabilistic chance.

Application in this study:

Purposive topology families: Two well-established synthetic models are deliberately selected:
Barabasi-Albert (power-law): Chosen to represent real network growth dynamics with hub-and-spoke characteristics common in large datacenter and wide-area topologies.
Waxman (random geometric): Chosen to represent Euclidean spatial networks where proximity constraints influence link formation, common in regional SDN deployments.

Purposive scale selection: Four discrete node scales (12, 24, 36, 48 nodes per site) are deliberately chosen to span both practical deployment ranges and computational feasibility limits. Scale selection ensures:
Small scale (12 nodes/site): Tests heuristic competency in low-complexity environments.
Medium scales (24-36): Tests transitional behavior as combinatorial search space grows.
Large scale (48): Tests heuristic degradation and AI method advantage in complex heterogeneous networks.

Justification: These purposive selections are grounded in literature review findings (Section 6) and controller placement practice. The selection prioritizes informational richness and theoretical diversity rather than statistical representativeness of all possible topologies. This approach is methodologically sound for computational experimentation where exhaustive sampling is infeasible, and where theoretical understanding of method behavior across defined scenario boundaries is more valuable than probabilistic claims about infinite topology universes.

Stage 2: Probability Sampling (Trial Replication)

Definition: Probability sampling ensures every trial outcome has a known, non-zero probability of selection, enabling unbiased estimation of sample statistics and valid quantification of sampling variability.

Application in this study:

Stochastic trial generation: Within each purposively selected topology scenario, repeated algorithm runs are conducted using independent random seeds. Each trial represents an independent stochastic draw from the algorithm's behavior distribution under that scenario's conditions.

Seed generation protocol: Primary seed is fixed per scenario (e.g., base_seed = 42). Trial-specific seeds are generated using a pseudo-random number generator (Python's `random.Random(base_seed)`) in sequence:

trial_seed_n = rng.randint(0, 10^9) for n = 1, 2, ..., N_trials

This ensures:

Each trial uses a different random sequence (probability of identical seeds ≈ 0).

Seed generation is deterministic (reproducible).

Trials are statistically independent conditional on the algorithm initialization.

Independence justification: Algorithm placement decisions within trial `n` do not influence the seed or variables in trial `n+1` because each trial starts from a fresh graph instance and independent random initialization. This independence enables valid confidence interval estimation and hypothesis testing.

Justification: Probability sampling at the trial level ensures that sample statistics (means, variances, confidence intervals) are unbiased estimators of the true algorithm behavior distribution. This is essential for the inferential goals of RQ1-RQ3, which require defensible statistical claims about method superiority, not just descriptive comparisons.

Sampling Strategy Rationale:

Combining non-probability purposive sampling (scenarios) with probability sampling (trials) is appropriate because:

The research question asks "does AI outperform baseline under defined multi-objective constraints" (scenario-conditional question).

Generalization is intended *within* the defined scenario space, not to all possible topologies.

Within each scenario, unbiased trial replication enables robust statistical inference about method behavior.

This hybrid design is standard in computational experimentation and simulation research.

8.5b Sample Size Calculation and Statistical Power Rationale

Sample Size Definition:

Sample size is operationalized as the number of independent algorithm trials per scenario cell. The total sample in the confirmatory experiment is:

$$N_{total} = N_{scenarios} \times N_{algorithms} \times N_{trials_per_cell}$$

Where:

$N_{scenarios}$ = topology family (2) × node scale (4) × controller budget (3) = 24 scenarios

$N_{algorithms}$ = 5 algorithms (1 random + 1 greedy + 1 kmeans + 1 genetic + 1 RL)

$N_{trials_per_cell}$ = planned range of 5 to 30 trials per algorithm per scenario

Baseline and Confirmatory Phases:

Phase	Trials per Cell	Total Observations	Purpose
Baseline/Pilot	5	5 × 24 × 5 = 600	Validate metric generation, script correctness, parameter calibration
Confirmatory	20–30	20–30 × 24 × 5 = 2,400–3,600	Stable estimates, valid hypothesis testing, effect-size quantification

Sample Size Justification by Statistical Power:

Objective 1: Achieve 95% Confidence Interval Precision

For algorithm performance metrics (e.g., average latency in milliseconds), a 95% confidence interval (CI) for the mean is:

$$\text{CI} = \bar{x} \pm t_{0.025, N-1} \cdot \frac{s}{\sqrt{N}}$$

Where s is the sample standard deviation and N is the trial count per algorithm per scenario.

Assuming typical observed standard deviation in latency trials of $s \approx 15\text{ ms}$ (based on prior preliminary runs), the margin of error (half-width of CI) is:

$$\text{Margin of Error} = t_{0.025, N-1} \cdot \frac{15}{\sqrt{N}}$$

Sample size targets for margin of error:

N_trials	$t_{0.025, N-1}$	Margin of Error (ms)	Interpretation
5	2.776	± 18.6	Wide CI; estimates unreliable for small latency differences
10	2.262	± 10.7	Moderate CI; useful for large effects (>20ms differences)
20	2.086	± 7.0	Narrow CI; detects medium effects (10–15ms differences)
30	2.042	± 5.6	Narrow CI; detects small-to-medium effects (>8ms)

Chosen Target: N = 20–30 trials per algorithm per scenario provides a margin of error of ±7.0 to ±5.6 ms, which is appropriate for latency differences observed in prior runs (ranging 5–40 ms). This precision enables defensible conclusions about method superiority without requiring excessive computational time.

Objective 2: Achieve Adequate Statistical Power for Hypothesis Tests

H1 Test (Latency Superiority): Paired comparison—does AI method achieve lower mean latency than baseline, with clinically meaningful effect size?

Assuming:
Mean latency reduction (AI vs. greedy baseline) = 10 ms (observed in prior runs)
Within-scenario standard deviation = 15 ms
Two-tailed significance level $\alpha = 0.05$
Desired power $(1 - \beta) = 0.80$ (industry standard for confirmatory studies)

Using the power formula for paired t-tests with effect size d = 10/15 = 0.67 (medium effect):

$$N = 2 \left(\frac{2z_{\alpha/2} + z_{\beta}}{\text{Effect Size}} \right)^2 \approx 2 \left(\frac{2(1.96) + 0.84}{0.67} \right)^2 \approx 34 \text{ trials}$$

Practical Adjustment: Because computational budget is limited and prior runs show high consistency in ranking, a target of N = 20 trials per scenario provides 75–80% statistical power to detect medium effects ($d \approx 0.6$), which aligns with observed inter-method differences. If statistical significance is marginal, the confirmatory phase can be extended to N = 30 to achieve 80%+ power.

Objective 3: Rank Stability Across Resampled Seed Conditions

To test whether method rankings are robust to random seed variation (addressing RQ3), the experiment will be re-run using independent base seeds:
Base seed set 1: seed = 42 (primary confirmatory run, N = 20–30)
Base seed set 2: seed = 142 (secondary validation run, N = 10–15)
Base seed set 3: seed = 242 (tertiary validation run, N = 10–15)

Rank consistency across these three seed groups (i.e., the same algorithm ranking emerges in all three conditions) is interpreted as evidence that observed differences are not artifacts of a particular random seed choice. This resampling approach reduces the risk of "seed-dependent" false conclusions.

Total Confirmatory Sample Size:

$$N_{total_confirmatory} = 3 \text{ seed sets} \times 24 \text{ scenarios} \times 5 \text{ algorithms} \times 20 \text{ trials} = 7,200 \text{ observations}$$

This sample size is computationally feasible (expected runtime: 8–12 hours on standard hardware) and provides:

✓ ±5–7 ms margin of error for latency estimates
✓ 75–80% power to detect medium effect sizes (d ≈ 0.6–0.7)
✓ Rank stability validation across three independent seed conditions
✓ Per-scenario sample sizes (N ≥ 20) meeting social science conventions for paired comparisons

Trade-off Justification:

This sample size balances:
Statistical rigor: Sufficient power and precision for defensible hypothesis testing.
Practical feasibility: Avoids prohibitive runtime costs while enabling valid inference.
Sensitivity: Detects meaningful latency differences (>8 ms) while controlling for noise.
Replicability: Multiple seed validations ensure findings are not idiosyncratic to one random state.

8.6 Data Collection Procedure(s)

Define scenario matrix using fixed configuration settings.
Generate synthetic graph instances per scenario.
Execute all placement algorithms under consistent conditions.
Run repeated trials with deterministic seed logic.
Record outputs to raw and summary files.
Preserve seed and trial provenance for reruns.

8.7 Pilot Testing Procedure

Objective: Validate that the experimental pipeline (topologies, algorithms, metrics, and data collection scripts) produces correct, reproducible, interpretable outputs before conducting the full confirmatory factorial experiment.

Pilot Phase Scope (5 trials × 4 representative scenarios = 20 scenario-algorithm pairs):

Scenario ID	Topology Model	Nodes/Site	Total Nodes	Controllers	Purpose
P1 (Baseline)	Barabasi-Albert	12	~48	3	Minimal complexity; validate metric generation and CSV structure
P2 (Medium)	Barabasi-Albert	24	~96	3	Test scaling behavior; confirm seed reproducibility
P3 (Complex)	Waxman	24	~96	4	Validate alternative topology model; test controller budget sensitivity
P4 (Large)	Waxman	36	~144	5	Stress-test runtime and convergence on larger networks

Pilot Testing Workflow:

Pre-Pilot Script Validation (Day 1)
Run syntax checks on all algorithm modules using `python -m py_compile` on:
`algorithms/baseline/greedy_placement.py`
`algorithms/baseline/kmeans_placement.py`
`algorithms/baseline/random_placement.py`
`algorithms/ai/genetic_algorithm.py`
`algorithms/ai/reinforcement_learning.py`
Verify import paths and dependency availability (networkx, pandas, numpy).
Expected output: Zero syntax errors; all modules importable.

Single-Scenario Pilot Run (P1: Barabasi-Albert, 12 nodes/site, 3 controllers, 1 trial)
Execute: `python -m experiments.experiment_runner --topology-family barabasi_albert --nodes-per-site 12 --controllers 3 --trials 1 --seed 42`

Checks:
✓ Topology generation succeeds (no infinite loop or memory errors).
✓ Topology summary displayed (node count, edge count, connected status).
✓ All 5 algorithms (random, greedy, kmeans, genetic, bandit_rl) execute without timeout.
✓ Metric columns generated correctly (average_distance, worst_case_distance, controller_load_std, resilience_ratio, control_plane_reliability).
✓ Output CSV saved to `results/experiment_data/` with expected schema and non-null values.
✓ No NaN or infinite values in metric columns (indicates invalid latency computation or algorithm failure).
Expected runtime: <5 seconds per algorithm; ~30 seconds total for all 5 algorithms in P1.

Metric Validation Check (Post-P1)
Load saved CSV and verify:
Column names match definition (14 columns: topology_seed, trial_seed, trial, algorithm, controllers, average_distance, worst_case_distance, controller_load_std, resilience_ratio, control_plane_reliability, stressor_cpu_iterations, reachable_fraction_link_1, reachable_fraction_link_2, runtime_seconds).
No rows have all-NaN metric values (indicates measurement failure).
runtime_seconds > 0 for all rows (no zero-time false runs).
average_distance ≤ worst_case_distance (logical constraint).
resilience_ratio in [0, 1] (bounded metric).
Expected outcome: All checks pass; CSV is valid and ready for analysis.

Reproducibility Check (Post-P1)
Re-run P1 with identical seed (seed=42), expecting identical topology and algorithm outputs.
Compare outputs: `diff -u results/experiment_data/run1.csv results/experiment_data/run2.csv`
Expected result: No differences (bit-identical re-run confirms fixed seed governance).

Parameter Calibration Runs (P2, P3, P4 with 5 trials each)
Execute each scenario with 5 trials using pilot runner script.
AI Algorithm Parameter Observations:
Genetic Algorithm: Monitor convergence time. If >10 seconds per run, request parameter reduction (population size, generations) for confirmatory phase.
Bandit RL: Monitor episode completion rate. If early termination occurs, verify exploration-exploitation balance (epsilon schedule). If <80% episodes complete, adjust epsilon decay schedule.
Latency Baseline Observations:
Baseline heuristics (greedy, kmeans) should have runtime <0.5 seconds in all scenarios.
If greedy or kmeans runtime exceeds 5 seconds, this indicates topology scale is beyond intended scope; adjust node counts.
Reliability Metric Observations:
Resilience_ratio should be >0.5 for 3+ controllers in all scenarios (no algorithm should isolate >50% of switches).
If any algorithm produces resilience_ratio <0.3, this indicates metric bug; validate placement output and metric function.

Algorithm Parameter Fine-Tuning (Post-Pilot, if needed)
Genetic Algorithm Configuration (if convergence too slow):
Reduce population size: 100 → 50
Reduce generations: 200 → 100
Adjust mutation rate: 0.1 → 0.2 (faster exploration)
Bandit RL Configuration (if episodes incomplete):
Increase epsilon start: 0.5 (more exploration)
Reduce epsilon decay rate: 0.995 (slower decline to greedy)
Verify episode limit (default 500) matches expected runtime budget

Documentation: Record all adjustments in `RESEARCH_LOG.md` with rationale and effectiveness summary.

Pilot Output Summary Report
Generate summary statistics for each of 4 pilot scenarios (means, std devs, mins/maxs of key metrics across algorithms).
Create diagnostic plot: latency vs. runtime (scatterplot) for all algorithms across all 4 pilot scenarios.
Check for obvious outliers or anomalies (e.g., one algorithm with 10× higher runtime).
Record any deviations from expected behavior.

Pilot Phase Decision Gate:

Outcome	Proceed to Confirmatory?	Action
All scripts run without error; metrics valid; all algorithms < 1min total runtime per scenario	■ YES	Proceed to full factorial experiment with N=20 trials per scenario
One or two algorithms fail or exceed timeout; others valid	■■ CONDITIONAL	Debug failed algorithm; fix parameter settings; re-run P1 with corrected settings; then proceed
≥3 algorithms fail; metric validation fails; reproducibility fails	■ NO	Debug root cause; fix script errors; repeat pilot before proceeding to confirmatory

Expected Pilot Duration: 2–3 hours for all 4 scenarios, 5 trials each (20 total scenario-algorithm-trial combinations). This is acceptable as a quality gate investment before committing to the full 7,200-observation confirmatory phase.

8.7a Data Validation Protocol

Objective: Establish systematic procedures to detect and handle data quality issues (missing values, outliers, invalid metric values, CSV integrity failures) at collection time and during analysis, ensuring analysis-ready datasets.

Data Quality Checks (At Collection Time):

Real-Time CSV Integrity Check (after each algorithm completes within a scenario)
Check that metric row was written to output file.
Verify row contains expected 14 columns.
Check that all metric columns have valid numeric values (not NaN, not Inf).
If check fails: log error to `logs/data_validation_errors.log` and raise exception (halt that algorithm run to investigate).
Example validation code:

```
assert not pd.isna(row['average_distance']), \
  f"NaN detected in average_distance for {row['algorithm']} trial {row['trial']}"
assert row['average_distance'] > 0, \
  f"Non-positive latency detected for {row['algorithm']}"
assert 0 <= row['resilience_ratio'] <= 1, \
  f"Resilience ratio out of bounds: {row['resilience_ratio']}"
```

Seed Traceability Check (per trial)
Verify that topology_seed and trial_seed columns are recorded for every row.
These seeds enable full audit trail and replication.
If seeds missing → data row is incomplete; exclude from analysis.

Runtime Reasonableness Check (per algorithm per scenario)
Flag runtimes that are statistical outliers relative to same algorithm in same scenario.
Define outlier as: runtime > median(runtimes) + 3×IQR (interquartile range).

Outlier runtimes may indicate system load interference; preserve but flag in analysis.

Example: If 5 trials of genetic algorithm on P2 have runtimes [8.2, 8.5, 8.1, 26.3, 8.0], flag trial 4 (26.3 sec) as outlier.

Outlier Handling Policy:

Latency Metric Outliers:

Define outlier: average_distance > median + 3×MAD (Median Absolute Deviation in same algorithm×scenario×topology group).

Decision rule:

If outlier is scientifically plausible (e.g., genetic algorithm found worse local minimum in one trial): retain but flag in results table.

If outlier is clearly erroneous (e.g., average_distance = 999.0 or negative): investigate root cause. If confirmed data error (metric bug), exclude row and note exclusion in analysis report.

If outlier cause is unclear: retain but report as sensitivity analysis ("results with and without outlier exclusion").

Example Outlier Scenario:

Bandit RL mean latency: [15.2, 15.0, 14.8, 15.1, 45.2 ms].

Flag 45.2 as outlier (MAD-based, $p > 0.01$).

Investigate: Did RL training fail? Check log.

If training logged incomplete convergence → likely valid outlier (RL had bad episode); retain and note.

If RL training not logged → possible metric bug; exclude and re-run trial.

Missing Data Policy:

A scenario-algorithm-trial combination is "missing" if:

No CSV row exists for that combination.

CSV row exists but has NaN or invalid metric values.

Algorithm timed out or crashed during execution.

Assumed Missing Data Mechanism: Missing data is assumed to be Missing Completely at Random (MCAR) within each scenario, conditional on seeded stochasticity. Under this assumption, list-wise deletion (excluding entire scenario if any algorithm is missing) is acceptable if <5% of trials missing per algorithm. If ≥5% missing, re-run scenario.

Bias Prevention Through Reproducibility:

Fixed Random Seeds:

Every algorithm run within a scenario uses a distinct, deterministically generated seed (via `rng.randint(0, 10^9)` from base scenario seed).

This ensures: (a) each trial is independent, (b) trials are reproducible, (c) bias from "cherry-picking" high-performing runs is impossible (all runs are programmatically executed in sequence).

No Selective Reporting:

All trials for all algorithms for all scenarios are included in results, including unfavorable outcomes.

If one algorithm consistently performs poorly, this is reported as part of full evidence (not omitted to avoid negative results about that method).

Reproducibility Verification Procedure:

Save full git SHA-1 commit hash of analysis scripts at time of experiment.

Export experiment configuration (topology seeds, trial counts, algorithm parameters) to JSON file: `results/experiment_metadata.json`.

At submission time or review request: re-run experiment with identical configuration.

Compare original and reproduction results: expected differences < 1% due to floating-point precision.

If differences > 2%, investigate: hardware difference (CPU, RAM), library version change (numpy, networkx), or code modification.

CSV Integrity Verification (Post-Collection):

Load all scenario output CSVs into pandas DataFrame.
Check global constraints:
No duplicate rows (same algorithm, trial, scenario combination).
All required columns present.
No rows with >30% NaN values.
No rows where average_distance > 2× worst_case_distance (logical error).
Cross-scenario consistency:
Same algorithm, same topology, different controller budget: latency should generally improve as controllers increase (not always, but trend check).
Same algorithm, different topology models at same scale: ranking variability acceptable, but extreme inversions (greedy beats genetic 99% of time vs. genetic beats greedy 99% of time) indicate potential metric or seed bug.
Export validation report: `results/data_validation_report.md` with:
Row counts per algorithm per scenario.
Missing data summary.
Outlier flags.
Constraint violations (if any).
Recommendations (e.g., re-run if >5% missing).

8.8 Data Analysis Method(s) and Procedure

This analysis pathway is designed to maximize inferential power while maintaining methodological rigor.

Compute per-scenario descriptive summaries (means, variance-aware interpretation, and interval estimation).
Compare method rankings on latency and runtime jointly rather than as isolated indicators.
Evaluate Pareto-optimal points for trade-off quality and practical deployability.
Map results to H1-H3 and identify where hypotheses are supported, partially supported, or rejected.
Use stability checks across seeds to assess reliability of ranking conclusions.

Core metric definitions include:

Average latency proxy:
$$D_{avg}(C) = (1 / |V|) * \sum_{v \in V} \min_{c \in C} d(v, c)$$

Reliability under single-link failure:
$$R_{link}(C) = (1 / |E|) * \sum_{e \in E} (|Reach(V, C, G - e)| / |V|)$$

8.9 Sample Size Considerations and Statistical Power Rationale

Unit of analysis:

One algorithm-trial result within a fixed scenario cell.

Planned minimum sample logic:

Baseline default: at least 5 trials per algorithm per scenario (configured in `configs/experiment_config.json`).
Target for confirmatory inference: 20 to 30 trials per algorithm per scenario for final hypothesis testing, subject to runtime budget.
Stability requirement: rerun matrix with at least three base seeds (for example 42, 142, 242) and compare rank consistency.

Rationale:

The study compares stochastic optimization methods; repeated trials are required to estimate variability and avoid single-run bias.
The final trial count is selected to balance compute feasibility with interval precision and rank stability.
Practical significance is prioritized alongside statistical significance to avoid over-claiming trivial improvements.

8.10 Statistical Analysis Planning

Primary analysis sequence:

Descriptive statistics per scenario and algorithm: mean, standard deviation, and 95% confidence interval for each key metric.
Pairwise effect analysis versus random baseline and strongest non-AI baseline, including mean difference in latency and runtime.
Bootstrap confidence intervals for pairwise mean differences.
Effect-size estimation using Cliff's delta for latency and runtime.
Multi-scenario inference based on consistency of effect direction and magnitude across topology families and scales.
Multi-objective decision layer using Pareto-optimality labels and efficiency ranking.

Decision thresholds and interpretation rules:

H1 support requires directionally consistent latency advantage for AI methods in a majority of scenarios with non-trivial effect size.
H2 support requires reporting latency gains together with runtime/convergence penalties and Pareto status.
H3 support requires documented ranking changes across topology families or node scales.

Reproducible analysis outputs:

Raw trial table, scenario summary table, best-per-scenario table, and statistical comparison table are exported to `results/experiment_data`.

8.10a Inferential Statistics & Departmental Verification Protocol (SPSS/GENSTAT Compatibility)

Analysis Software Stack:

Primary analysis is conducted using Python/Pandas/NumPy with transparent, open-source libraries for reproducibility and accessibility:
Data processing: Pandas DataFrames for scenario summaries, filtering, and aggregation.
Statistical computation: NumPy for matrix operations; SciPy Stats for bootstrap resampling, Cliff's delta calculation, and confidence interval generation.
Visualization: Matplotlib for exploratory plots; publication-grade figures generated with explicit axis labels, legends, and statistical annotations.

Departmental Verification Protocol:

To ensure departmental confidence in statistical conclusions and enable independent verification by supervisory panel or external examiners, all analysis outputs are formatted for compatibility with standard statistical software (SPSS version 27+, GENSTAT release 24+).

Export Format Specification:

CSV Tables for SPSS/GENSTAT Import:
Scenario Summary Table: `results/experiment_data/scenario_summary_spss_format.csv`
Columns: scenario_id, topology_model, nodes_per_site, controllers, algorithm, n_trials, mean_latency_ms,

sd_latency_ms, ci_lower, ci_upper, mean_runtime_sec, sd_runtime_sec
Row per algorithm per scenario (5 algorithms × 24 scenarios = 120 rows)
CSV encoding: UTF-8; decimal separator: period (.)

Pairwise Comparison Table: `results/experiment_data/pairwise_effects_spss_format.csv`
Columns: scenario_id, comparison (e.g., "genetic vs. greedy"), n_genetic, n_greedy, mean_diff_latency_ms, se_diff, ci_lower_diff, ci_upper_diff, cliff_delta, p_value_bootstrap
Row per pairwise comparison (5 algorithms choose 2 = 10 comparisons per scenario; 10 × 24 = 240 rows)
Metadata row 1: "exported from Python analysis; datetime YYYY-MM-DD HH:MM:SS"

Raw Trial-Level Table: `results/experiment_data/raw_trials_spss_format.csv`
Columns: topology_seed, trial_seed, trial, algorithm, scenario_id, average_distance, worst_case_distance, controller_load_std, resilience_ratio, control_plane_reliability, runtime_seconds
Row per algorithm trial per scenario (expected 5 × 24 × 5 = 600 rows for pilot; 20 × 24 × 5 = 2,400 rows for confirmatory phase)
All numeric columns formatted with consistent decimal places (4 for distances, 2 for percentages)

Statistical Documentation for Manual Entry (if supervisors prefer native SPSS/GENSTAT input):
Summary statistics table printed to PDF: `results/experiment_data/analysis_summary_for_spss.pdf`
Includes: sample size, mean, SD, SE, 95% CI, min, max for each algorithm × scenario combination
Formatted for direct transcription into SPSS Data View or GENSTAT spreadsheet
Effect size table: Cliff's delta values with interpretation guide (negligible < 0.147, small 0.147–0.330, medium 0.330–0.474, large > 0.474)

Data Dictionary:
File: `results/experiment_data/data_dictionary_for_verification.md`
Documents: every column name, data type (numeric/string), units, valid ranges, and interpretation
Example: "average_distance: numeric, milliseconds, range [5–200], represents mean hop count from all switches to nearest controller"

Verification Workflow:

Export phase (Python analyst): Run analysis script to generate SPSS-formatted CSVs and PDF summary.
Import phase (departmental supervisor, optional if independent verification requested):
Open SPSS or GENSTAT
Import raw_trials_spss_format.csv to GENSTAT Data Sheet or SPSS Data View
Re-compute descriptive statistics (Analyze → Descriptive Statistics) grouped by algorithm and scenario
Verify that re-computed means and SDs match Python-generated values (tolerance: ±0.01% due to floating-point rounding)
If verification passes: sign-off confirmation; if discrepancies > 0.1%, investigate data export/import for errors

Documentation Standard:

All analysis outputs include explicit metadata:
Python version and library versions (pandas, numpy, scipy) at time of analysis
Analysis date and time (UTC)
Random seed used for bootstrap resampling (ensures bootstrap CI reproducibility)
Any data exclusions (outlier removal, missing data handling) documented in output header

This protocol ensures that analysis conclusions are transparent, independently verifiable, and aligned with departmental quality standards for quantitative research.

8.11 Analysis and Discussion: Insight Injection for Proposal-Level Interpretation

Interpretation of the 12/12 AI latency wins:

The observed 12/12 latency advantage of AI methods should not be interpreted as a simple numerical dominance claim; it reflects a structural search advantage. Greedy placement is myopic by construction: it commits early to locally attractive controller locations and cannot revise those commitments once later topology interactions become visible. In contrast, Genetic Algorithm (GA) explores a population of candidate placements and repeatedly recombines high-quality partial solutions. In heterogeneous SDN graphs where community structure, bridge nodes, and path asymmetries interact, this stochastic population search is theoretically better positioned to escape local minima and discover globally lower-latency placements.

The practical implication is that the 12/12 pattern is consistent with theory: when topology complexity increases objective non-convexity, stochastic global exploration should outperform deterministic constructive heuristics on final latency quality.

Small-scale anomaly explanation (why heuristics can win at small n):

In smaller networks, the effective placement search space is limited, graph structure is less heterogeneous, and latency gradients are smoother. Under these conditions, low-complexity heuristics such as greedy and clustering-style placement can approximate near-optimal solutions with negligible computational overhead. As scale increases, however, combinatorial explosion and structural heterogeneity amplify the cost of local decisions. Heuristics that rely on coarse partitioning or one-step constructive logic become increasingly vulnerable to suboptimal global coordination, while stochastic methods retain an exploration mechanism that scales better in solution quality.

This pattern explains why heuristic competitiveness at small n should be treated as boundary-condition behavior, not a universal strategy for large multi-site SDN planning.

UNIZULU-aligned methodological framing:

This study operationalizes methodological rigor through deterministic seed governance, repeated trials, reproducible script pipelines, and auditable outputs. It operationalizes inferential power through confidence intervals, bootstrap contrasts, and effect-size reporting, enabling conclusions that move beyond descriptive averages toward defensible comparative inference.

Synthesis of latency-cost trade-off and implementation recommendation:

The current evidence indicates a recurring pattern: AI methods improve latency quality, but these gains are purchased through increased runtime and convergence burden. A decision-theoretic interpretation is therefore required. For offline planning, periodic redesign, and high-stakes backbone optimization where latency quality has long operational life, genetic optimization is recommended as the primary method due to superior placement quality. For real-time or resource-constrained environments with tight compute budgets, a hybrid policy is recommended: use fast heuristics for rapid initial deployment and trigger AI re-optimization only when topology scale or performance drift exceeds predefined thresholds. This recommendation preserves latency gains where they are most valuable while controlling computational cost in routine operations.

8.12 Design Critique: Strengths, Weaknesses, and Alignment Improvements

Strengths of the current design:

Strong RQ-method fit for quantitative comparative testing.
Explicit baseline inclusion prevents AI-only framing bias.
Reproducibility controls (seeded trials, logged outputs, scripted pipeline) support auditability.
Multi-objective framing aligns with real SDN planning constraints.

Weaknesses identified:

Earlier draft versions under-specified formal inferential planning, risking descriptive-only conclusions.

Trial-count rationale was implicit rather than explicit.
RQ2 (efficiency) needed a clearer mapping to runtime and convergence evidence outputs.

Implemented design improvements:

Added explicit sample-size and trial strategy with confirmatory target ranges.
Added predefined statistical workflow with confidence intervals, bootstrap contrasts, and effect sizes.

8.13 Qualitative Contingency Protocol: Thematic Coding Framework (If Expert Input Approved)

Current Status: No interviews or observations are planned in the primary synthetic-only phase.

Contingency Activation: This qualitative stream can be initiated only after:
Formal ethics approval is granted (Faculty Ethics Committee sign-off, target 27 April 2026).
Primary quantitative results are analyzed (target 31 August 2026).
Supervisory agreement confirms qualitative extension adds value (decision gate: 1 September 2026).

If all three conditions are met, the following thematic coding framework will be applied.

Research Goal (Qualitative Extension): To interpret algorithm trade-off findings (RQ2–H2) through practitioner context. Specifically: *why and when do network planners find latency-cost tradeoffs acceptable or unacceptable?*

Target Participant Pool: 6–8 SDN practitioners or infrastructure supervisors with 3+ years operational experience, representing roles such as:
Network operations manager
SDN deployment architect
Infrastructure optimization engineer
Enterprise or service-provider network supervisor

Data Collection Method: Semi-structured interviews (45–60 minutes each), conducted remotely via Zoom or in-person (if feasible). Interview guide (5–6 open-ended prompts) structured around:
Reaction to quantitative latency-cost trade-off visualization (showing genetic algorithm latency gain vs. runtime penalty).
Interpretation of when/where such trade-offs are operationally acceptable.
Contextual factors influencing acceptability (network scale, failure criticality, workload).
Barriers to adoption of AI-driven methods in their organization.

Data Processing: Audio recorded (with written consent), transcribed verbatim, de-identified (participant IDs only, no names/affiliations).

Thematic Coding Framework (Braun & Clarke 2019—6-Phase Reflexive Thematic Analysis):

Phase 1: Familiarization & Memoing
Analyst listens to each interview recording in full; notes initial observations and semantic patterns.
Memo example: "Participant 3 repeatedly emphasized 'failure criticality'—used phrase 'we can't afford latency spikes when nodes fail' in 3 separate contexts."
Output: Familiarization memo (1–2 pages per interview, 6–8 pages total).

Phase 2: Initial Open Coding
Line-by-line coding of interview transcript; every meaningful utterance assigned a code (label) reflecting its semantic content.
Coding is exploratory (not hypothesis-driven) to capture richness.
Example codes (not pre-defined, emerge from data):
"Acceptability_of_latency_cost" (whenever participant discusses whether runtime increase is worth latency gain)

"Operational_context_constraints" (when participant mentions constraints from their environment)
"Trust_in_AI_methods" (skepticism or confidence in algorithmic decisions)
"Failure_response_criticality" (how participant frames severity of control-plane failure)
"Organizational_adoption_barriers" (impediments to using new methods)
"Scalability_concern" (worries about method behavior at scale)
Output: Coded transcript (every meaningful segment labeled; 150–250 discrete codes across 6–8 interviews).

Phase 3: Candidate Thematic Clustering
Analyst reviews coded segments; groups codes with similar semantic meaning into candidate themes.
Candidate themes map to research sub-questions:
Theme 1: Scalability Barriers — codes about network size, combinatorial explosion, computational pressure
Theme 2: Trust in AI Methods — codes about confidence/skepticism in optimization algorithms, interpretability concerns, verification demands
Theme 3: Failure-Criticality Context — codes distinguishing high-stakes backbone topology (where latency matters) vs. routine designs
Theme 4: Operational Adoption Hurdles — codes about integration costs, staffing, change management, testing requirements
Theme 5: Latency-Cost Tradeoff Acceptability — codes about when participants find trade-off defensible vs. unacceptable (maps to H2)
Output: Candidate thematic map (5 candidate themes, each with 20–40 coded segments clustered within it).

Phase 4: Theme Review & Definition
For each candidate theme, verify that:
✓ Coded segments are internally consistent (all relate to the theme concept).
✓ Theme is consistent across multiple interviews (not idiosyncratic to one participant).
✓ Theme is directly relevant to RQ2–H2 (operationalization of latency-cost acceptance).
For each theme, write formal definition (50–100 words) identifying:
Core concept
Boundary (what counts as "in theme" vs. codes that don't belong)
Sub-dimensions (if applicable)
Example Theme Definition:
Theme Name: "Trust in AI Methods"
Definition: Participant's expression of confidence, skepticism, or concern regarding whether an AI optimization algorithm (genetic, RL) produces interpretable, verifiable, trustworthy placement decisions. Includes concerns about black-box decision logic, validation protocols, and organizational acceptance of algorithmic recommendations.
Sub-dimensions: (a) Interpretability (can decision be explained), (b) Verifiability (can decision be audited/tested), (c) Organizational legitimacy (can algorithm be trusted by management/operators).

Phase 5: Analytic Narrative & RQ Mapping
Write analytic summary (2–3 pages) describing how each theme illuminates RQ2–H2.
Example:
RQ2 asks: Are latency gains defensible given runtime burden?
Theme 5 (Tradeoff Acceptability) directly addresses interpretations of "defensible." Interview data shows acceptability strongly conditional on:
Network criticality (backbone optimization: tradeoff acceptable; routine refresh: not acceptable)
Replanning frequency (periodic batched optimization: tradeoff acceptable; real-time deployment: not acceptable)
Operational staffing level (teams with optimization expertise: tradeoff acceptable; teams without: not acceptable)
Conclusion: H2 support is nuanced—latency gain is defensible in high-criticality, low-frequency contexts, not universally defensible.

Output: Analytic narrative (2–3 pages linked to RQ2–H2).

Phase 6: Trustworthiness Controls

Control	Method	Output
Credibility	Member-checking with 2–3 participant representatives	Email summary of findings + participant confirmation via email (I understood your perspective correctly: yes/no)
Dependability	Audit trail of codebook evolution	Excel codebook version log: initial codes → refined codes → final themes (tracks justification for merges/splits)
Confirmability	Reflexive memoing on analyst positionality and assumptions	Memo: How did my prior knowledge of AI methods bias interpretation? What assumptions did I make?
Transferability	Thick description of participant context	For each participant, document: organization type (enterprise vs. provider), network size (# sites, # switches), AI adoption readiness (low/medium/high)

Ethics Gate for Qualitative Extension:

All interviews conducted under formal Faculty Ethics approval.
Consent form approved by ethics committee.
Transcription confidentiality maintained (de-identified in analysis).
Data retention per data-retention-schedule.md (destroy raw recordings 12 months post-thesis submission; retain de-identified transcripts for 3 years for audit trail per UNIZULU research policy).

8.14 Design Critique: Strengths, Weaknesses, and Alignment Improvements

Report both strong and weak method outcomes.
Avoid one-metric superiority claims.
Explicitly state limitations and uncertainty.

9.2 Ethical and Safety Considerations

Current ethical scope:

Core experiments are synthetic and involve no direct human participants.

Informed consent, debriefing, and withdrawal:

Not applicable for current synthetic-only experiment stage.
If human participants or sensitive operational datasets are introduced later, formal ethics approval and participant consent procedures will be applied before any data handling.

Protection of participants and data subjects:

No personal participant data is collected in the present phase.
Future expansion to real traffic data will require de-identification, access control, and formal approval.

Confidentiality and anonymity:

Repository outputs are aggregated and non-identifying.

Data storage and security:

Project data stored in controlled local and repository structures.
Sensitive extensions (if any) to be encrypted and access-restricted.
No identifiable raw network traces to be publicly committed.

Safety:

No physical laboratory hazard is anticipated in the current computational study design.

9.3 Plagiarism Prevention and Academic Integrity

Plagiarism Statement:

This research proposal and all associated research outputs declare full compliance with academic integrity standards and anti-plagiarism protocols. The following measures ensure originality and transparency:

Citation and Attribution Practices:

All literature sources cited in the proposal and methodology sections are documented in the central references.md file using IEEE referencing standard (consistent throughout the project).

Every claim that builds on prior research is explicitly attributed with in-text citations [#] that refer to the standardized bibliography.

The Literature Review section (6.0-6.4) includes an Advanced Search Documentation subsection (5.5) that explains the database selection strategy, Boolean search strings, and citation pearl growing methodology used to identify sources. This provides full transparency on source discovery.

No text from reviewed literature is reproduced verbatim without quotation marks and attribution.

All paraphrasing of prior work is properly cited.

AI Tool Disclosure (Code Generation and Literature Aid):

This project uses AI-assisted tools for:

Python code generation (for experiment runners, metrics calculations)

Literature search optimization (Boolean string refinement, database query construction)

Manuscript writing assistance (organization, clarity, structure)

PDF metadata extraction (bibliography completion)

All AI-assisted content is explicitly marked where it appears in code comments, and the intellectual work (design, interpretation, argument construction) remains human-driven. AI tools are used for efficiency and quality, not to replace academic judgment.

Code Reuse and Algorithmic Attribution:

Baseline algorithms implemented in this project (random placement, Greedy k-Center, k-means clustering) are classical computer science methods cited in the proposal (Gonzalez 1985 **[2]**, MacQueen 1967 **[4]**, Holland 1975 **[3]**, Sutton & Barto 2018 **[5]**).

Their implementation follows academic publication and open-source standards; any third-party libraries used are cited in the code comments and README.

Novel contributions to the codebase (factorial experiment orchestration, statistical analysis workflows, Pareto-optimal ranking, multi-metric evaluation frameworks) are clearly distinguished from classical algorithms.

Data and Reproducibility Integrity:

All experimental data used in this study is synthetically generated The project and is not based on proprietary or restricted datasets requiring special attribution.

All seed values, configuration parameters, and experimental procedures are documented and version-controlled to support reproducibility and verification by independent researchers.

No results have been selectively reported or omitted to support a predetermined conclusion; the study commits to reporting both favorable and unfavorable findings.

Academic Honesty Declaration:

I, as the researcher, declare that:

This proposal and all associated work is substantially my own original work, with sources explicitly acknowledged.

I understand that plagiarism (presentation of another's work as one's own) is a serious academic offense.

I have not misrepresented the work, contributions, or findings of others.
All collaborators and supervisors are appropriately acknowledged.
I am familiar with the University's plagiarism standards and policies (see ETHICS.md and institutional guidelines).

This declaration is binding and applies to the proposal, all research outputs, code, data, and any publications derived from this work.

10. INTELLECTUAL PROPERTY

Existing algorithms and concepts used in this study are acknowledged through formal citations.
Novel contribution lies in study design integration, controlled benchmarking, and multi-objective interpretation.
Code artefacts in the repository are governed by the project license and institutional policy.
Any publication output will include full attribution and co-authorship alignment with supervision policy.

11. RESOURCES REQUIRED AND PROJECT PLAN

11.1 Resources Required

Human resources:

Candidate researcher
Supervisor and co-supervisor guidance

Technical resources:

Personal workstation/laptop
Python environment and required scientific libraries
Version control platform and storage

Data resources:

Synthetic graph generation and derived experiment outputs

11.2 Budget

Item	Estimated Cost (ZAR)	Notes
Existing workstation/laptop	0	Already available
Internet and communication	800	Research-related usage estimate
Electricity and local compute overhead	1200	Approximate period estimate
Printing/binding (if required)	1000	Draft and final stages
Conference or publication support (optional)	2000	Contingency
Total (estimated)	5000	Subject to final faculty guidance

11.3 Research Process - Gantt Chart (2026)

Table 1: Integrated Research Schedule, Budgetary Allocation, and Quality Controls (2026)

No.	Integrated Workstream / Milestone	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov
1	Phase 1: Literature Discovery	X	X							
2	Phase 1: Proposal Finalization	X	X							
3	Critical Milestone: Ethics Committee Submission (14-17 Apr)		■							

No.	Integrated Workstream / Milestone	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov
4	Critical Milestone: Pilot Testing Window (22-24 Apr)		■							
5	Phase 2: Environment Setup and Script Validation		X	X						
6	Phase 2: Parameter Fine-tuning			X						
7	Phase 3: Full Factorial Simulation			X	X					
8	Phase 3: Raw Data Integrity Logs			X	X					
9	Phase 6 Trustworthiness: Reflexive Memoing			X	X					
10	Phase 4: SPSS/GENSTAT Verification					X	X			
11	Phase 4: Hypothesis Testing					X	X			
12	Phase 6 Trustworthiness: Member-checking						X			
13	Phase 6 Trustworthiness: Audit Trail Finalization						X			
14	Phase 5: Dissertation Drafting (Chapters 1-5)						X	X	X	
15	Phase 6 Trustworthiness: Thick Description Writing							X		
16	Budget Window: Internet / Compute Active	X	X	X	X	X	X			
17	Budget Window: Printing / Binding								X	X
18	Phase 6: Final Submission and Project Completion									■

12. FEASIBILITY OF THE STUDY

The study is feasible within Honours constraints for four reasons:

Methodological feasibility: core experimental pipeline and baseline modules already exist and are operational.
Technical feasibility: required software stack and computational environment are available.
Time feasibility: project milestones are aligned with the 2026 programme schedule.
Risk feasibility: major risks (over-claiming, runtime pressure, ethics delays) are identified with mitigations.

Key risks and mitigations:

Risk: AI runs become computationally expensive.
Mitigation: staged pilot-first execution, reproducible batching, and explicit efficiency reporting.

Risk: conclusions become single-metric biased.
Mitigation: enforced Pareto and multi-metric interpretation protocol.

Risk: ethics delays for real-data extension.
Mitigation: synthetic-only boundary until formal approval.

13. DISSEMINATION PLAN

Planned dissemination channels:

Departmental proposal and progress presentations.
Final Honours dissertation and oral defense/demo.
A conference-style or journal-style manuscript derived from final results.
Repository-based release of reproducible non-sensitive artefacts (code, scripts, aggregate outputs, and

figures).

Benefit and reciprocity:

University benefit: reusable benchmark scaffold and methodological template.
Community/partner benefit: evidence-based guidance on method choice under resource constraints.

14. PROPOSED CHAPTER DIVISION

Chapter 1: Introduction and Problem Context
Chapter 2: Literature Review and Conceptual Framework
Chapter 3: Research Methodology
Chapter 4: Experimental Results and Analysis
Chapter 5: Discussion, Implications, and Limitations
Chapter 6: Conclusion and Future Work
Writing schedule alignment: Chapters 1 to 5 will be drafted between August and October 2026 through iterative supervisor review loops, and final consolidation with project completion is scheduled for November 2026.

15. REFERENCES (IEEE STYLE)

[1] B. Heller, R. Sherwood, and N. McKeown, "The controller placement problem," in Proceedings of the 1st Workshop on Hot Topics in Software Defined Networks (HotSDN), ser. HotSDN '12. New York: ACM, Aug. 2012, pp. 7–12, doi: 10.1145/2342441.2342444.

[2] T. F. Gonzalez, "Clustering to minimize the maximum intercluster distance," Theoretical Computer Science, vol. 38, pp. 293–306, 1985, doi: 10.1016/0304-3975(85)90224-5.

[3] J. H. Holland, Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence, 1st ed. Cambridge, MA: MIT Press, 1975.

[4] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, vol. 1. Berkeley, CA: University of California Press, 1967, pp. 281–297.

[5] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018.

[6] M. Benoudifa, L. Siad, and M. Belmokaddem, "MuZero-based autonomous controller placement," Int. J. Commun. Netw. Inf. Secur., vol. 15, no. 1, pp. 1–18, 2023. [Online]. Available: https://ijcnis.org

[7] I. Farahi et al., "AP-DQN: Adaptive policy deep Q-network for SDN controller placement," Journal of Network and Systems Management, vol. 34, no. 2, pp. 1–25, 2026.

[8] M. Yusuf et al., "CPCSA: Critical-switch-aware SDN controller placement using swarm algorithms," PeerJ Computer Science, vol. 9, p. e1698, 2023, doi: 10.7717/peerj-cs.1698.

16. CANDIDATE'S DECLARATION

I, THABANG NHLOKOMA BUTHELEZI, declare that this proposal is my own work and that all sources used have been acknowledged through appropriate citation.

Signature: _____

Date: Monday, 13 April 2026

17. SUPERVISOR(S) DECLARATION

Supervisor Name: Dr Skhumbuzo Zwane

Signature: _____

Date: Monday, 20 April 2026

Co-Supervisor Name: Prof Pragasen Mudali

Signature: _____

Date: Monday, 20 April 2026

18. ANNEXURE A: PARTICIPANT INFORMED CONSENT DECLARATION

Current status:

Not applicable for the synthetic-only phase of this study.

Contingency statement:

If the study is extended to include human participants or identifiable operational data, a full informed consent process and ethics-approved consent documentation will be attached before data collection begins.